Reproducible ML Pipeline for Chicago Taxi Trips 2024 using Dagster

GitHub Repository: https://github.com/gauriklad/dagster-ML-pipeline

Introduction: Conventional data-science processes often rely on monolithic Jupyter Notebooks. However, it fails reproducibility, causing the process to be less efficient. Even a single change to a model setup typically requires re-running of the whole notebook, which re-reads unnecessary data and reprocesses it. To attain reproducibility and computational efficiency, traditional process of working with a notebook can be transformed into an asset-based pipeline and run in Dagster.

Methodology:

The pipeline was designed in the form of decoupled assets, and the linear notebook logic was changed into a Directed Acyclic Graph (DAG):

- load_data :- ingests the raw Chicago Taci 2024 csv dataset.
- preprocess_data :- data cleaning step that drops nulls and performs the train/test split
- train_models :- concurrent execution of the 3 regression models, namely, Decision Tree, Random Forest and Linear Regression.
- compare_models :- estimating performance of all the models using mean absolute error (MAE) to determine the best one.

Efficiency Analysis:

In order to measure the benefit of the Dagster pipeline, a baseline "Full-Run" was compared with a "Partial Re-run", in which only the configuration of Random Forest model was changed.

- Scenario: Optimization of the max_depth hyperparameter in Random Forest Model on the dataset.
- Baseline (Full Re-run): 58 seconds
    - It was observed that the system was forced to re-load and re-process the csv file, consequently using unnecessary resources.
- Dagster (Partial Re-run): 19 seconds
    - Dagster's memorization engine found that the load_data and preprocess_data assets did not change and hence, fetched these outputs from storage and only model-training operations were run.
- Net Improvement: 67% reduction of execution time

Workflow Time Comparison Table

Feature	Jupyter Notebook	Dagster Pipeline
Full Pipeline Run	58 sec	58 sec
Single Model Tuning	58 sec	19 sec
Data Lineage	Manual	Automated
Reproducibility	Low	High